Zephyr Project

IT-Schulungen.com
Technology & Transformation

5.03 Build- und Image-Artefakte

ELF · BIN · HEX · Map · Symbole · signierte Images

5.03

Vier Kernartefakte im Buildverzeichnis

PFAD

`build/zephyr/`

`zephyr.elf`	gelinktes Programm mit Sections, Symbolen und Debug-Informationen
`zephyr.bin`	lineares Binärabbild ohne eingebettete Zieladressen
`zephyr.hex`	adressierte Intel-HEX-Datensätze für Flashwerkzeuge
`zephyr.map`	Linkerbericht zu Symbolen, Regionen und Speicherbelegung

Welche Dateien erzeugt werden, hängt von Board und Buildintegration ab.

ELF ist die technische Referenz

ANALYSE

```
readelf -h build/zephyr/zephyr.elf
readelf -S build/zephyr/zephyr.elf
nm -n build/zephyr/zephyr.elf
```

01 Sections

zeigen Code-, Daten-, BSS- und Metadatenbereiche.

02 Symbole

verbinden Adressen mit Funktionen und Objekten.

03 Entry Point

markiert den vom Image vorgesehenen Einsprung.

ELF enthält Analyseinformationen, die in BIN oder HEX teilweise fehlen.

ELF, BIN und HEX im Vergleich

01	`ELF`	vollständiges Linkergebnis für Debugging und Analyse
02	`BIN`	kompakter Bytebereich; Ladeadresse kommt aus dem Flashkontext
03	`HEX`	Daten plus Zieladressen in textbasierten Datensätzen

Das Format muss zum Bootloader, Programmer und Updatepfad passen.

Vom Quellcode zum Firmware-Image

Fünf Verarbeitungsschritte mit unterschiedlichen Ergebnissen

01

Kompilieren

Quellen werden Objekte

02

Archivieren

Bibliotheken bündeln

03

Linken

ELF und Map entstehen

04

Konvertieren

BIN und HEX ableiten

05

Signieren

freigegebenes Image erzeugen

Post-Processing verändert das Auslieferungsimage, nicht den Quellcode.

Welches Artefakt beantwortet welche Frage?

Wo liegt der Entry Point?	ELF-Header mit readelf auswerten
Welche Adresse hat ein Symbol?	ELF-Symboltabelle mit nm oder readelf lesen
Welche Flashadressen werden beschrieben?	HEX-Datensätze oder Runner-Konfiguration prüfen
Wie groß ist der rohe Payload?	BIN-Dateigröße und Slotgrenzen vergleichen
Wo landen Code und Daten?	Map-Datei und ELF-Sections gegenüberstellen

Die Fragestellung bestimmt das Werkzeug und das auszuwertende Artefakt.

Unsigniertes und signiertes Image unterscheiden

POST-PROCESSING

```
zephyr.bin → west sign / imgtool → zephyr.signed.bin
```

01 **Unsigniert**

reines Buildresultat
noch ohne Bootloader-Signatur

02 **Signiert**

Header, Hash und Signatur
für die Prüfung durch den
Bootloader

Flash- und Updatepfad müssen die erwartete Imagevariante verwenden.

Speicherverbrauch richtig lesen

FLASH

Persistent

.text und .rodata

initiale Werte aus .data

Image-Metadaten

RAM

Zur Laufzeit

.data wird kopiert

.bss wird genullt

Stacks und Heaps

ANALYSE

Zugeordnet

ELF-Sections

Map-Datei

Größenbericht

Dateigröße, Flashbelegung und RAM-Verbrauch sind unterschiedliche Messgrößen.

Buildartefakt, Release-Image und Paket unterscheiden

BUILD

Technische Ausgabe

ELF, BIN, HEX und Map entstehen aus dem Build

RELEASE

Freigegebenes Image

Version, Signatur und Ziel sind eindeutig festgelegt

PAKET

Transportierbare Einheit

Image plus Manifest, Prüfsumme und Metadaten

Nicht jedes Buildartefakt ist automatisch ein auslieferbares Firmware-Image.

Technische Zuordnung zum Quellstand

IDENTITÄT

```
VERSION + BUILD-ID + GIT-REVISION
TOOLCHAIN + ZEPHYR-REVISION + KONFIGURATION
```

01 Nachvollziehbar

Image und Analyseartefakte lassen sich demselben Build zuordnen.

02 Unverwechselbar

Dateiname, Version und Prüfsumme bezeichnen genau ein Image.

Eine Firmwarefreigabe verbindet Binärdaten, Herkunft und Integritätsnachweis.